

Exqutor 관련 References 24편 통합 요약

범위: References [1]~[24] 중 24편의 핵심 내용 요약

[1] AnalyticDB-V; A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data 총정리

역할군: (A) VAQ 개념 원조: 본 논문 문제 정의의 출발점

AnalyticDB-V는 Exqutor가 풀려는 문제, 즉 "벡터 검색과 SQL 분석 쿼리를 하나의 시스템 안에서 효율적으로 처리하는 것"을 최초로 산업 규모에서 구현한 시스템이다. 현대 데이터 파이프라인에서 구조화 데이터(테이블)와 비구조화 데이터(이미지, 텍스트 등)가 함께 존재하는데, 기존에는 이 두 종류를 별개의 시스템으로 처리해야 했다. AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자로 구현**하는 것으로, 사용자가 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있도록 한다.

시스템은 Lambda 아키텍처를 채택하여 스트리밍 레이어와 배치 레이어를 분리하고, VGPQ(Vector Graph Product Quantization)라는 새로운 ANN 검색 알고리즘을 제안한다. 가장 중요한 기여는 **정확도 인식 비용 기반 최적화(Accuracy-Aware Cost-Based Optimization)**로, ANN 검색의 본질적 근사성을 옵티마이저에 통합하여 정확도 요구사항과 비용 간의 트레이드오프를 관리한다. 다만 **카디널리티 추정** 문제를 깊이 다루지 않았으며, 이것이 Exqutor가 보완하는 핵심 빈틈이다. 본 논문과의 관계: VAQ(Vector-Augmented Analytical Query) 개념의 원형을 제시했으나, "벡터 검색 결과가 정확히 몇 건이나 나올지 추정하는 문제"는 미해결로 남겨두어, Exqutor의 근본적 동기를 제공한다.

현대 데이터 파이프라인에서는 **구조화 데이터**(매출, 재고, 사용자 정보 등 숫자/텍스트 테이블)와 **비구조화 데이터**(이미지, 동영상, 오디오, 텍스트 문서 등)가 함께 생성된다.

기존에는 이 두 종류의 데이터를 별개의 시스템으로 처리했다:

- 구조화 데이터 → 관계형 데이터베이스 (MySQL, PostgreSQL 등)
- 비구조화 데이터 → 별도의 벡터 검색 엔진 (Faiss 등)

문제는 실제 비즈니스에서 두 데이터를 **함께** 분석해야 하는 경우가 매우 많다는 것이다. 예를 들어:

- "이 상품 이미지와 비슷한 상품을 찾되, 가격이 100만원 이하이고 재고가 있는 것만 보여줘"
- "이 얼굴 사진과 유사한 사용자를 찾아서, 그 사용자의 최근 구매 이력을 분석해줘"

이런 쿼리를 처리하려면 개발자가 두 시스템 사이에서 수동으로 데이터를 옮기고 결합해야 했다. 이 과정이 매우 복잡하고 느렸으며, 최적화 기회를 완전히 놓치게 된다.

AnalyticDB-V는 **Lambda 아키텍처**를 채택한다:

스트리밍 레이어(Streaming Layer): 실시간으로 들어오는 벡터 데이터를 처리하며, 새로 삽입된 벡터는 즉시 검색 가능하다.

배치 레이어(Batching Layer): 주기적으로 새로 삽입된 벡터들을 압축하고, ANNS 인덱스를 재구축한다.

[2] Hybrid RAG-Empowered Multi-Modal LLM for Secure Data Management in Internet of Medical Things 총정리

핵심 문제는 의료 IoT 환경에서 X-ray 이미지, 심전도 시계열, 텍스트 진료 기록 같은 멀티모달 데이터를 통합 분석해야 한다는 것이다. 논문은 세 가지 계층으로 구성된 시스템을 제안한다: (1) 멀티모달 하이브리드 RAG로 각 모달리티별 후보를 검색하고 교차 검증, (2) 블록체인 기반 안전한 데이터 공유, (3) AoI(Age of Information) 메트릭으로 데이터 신선도를 보장하는 인센티브 메커니즘. 특히 첫 번째 계층의 RAG 파이프라인이 바로 **복합 VAQ의 실체**이며, 정확한 카디널리티 추정이 없을 때 얼마나 비효율적인지를 보여준다.

본 논문과의 관계: VAQ의 실제 구현에서 벡터 유사도 검색과 메타데이터 필터링, 다중 모달리티 교차 검증이 결합될 때, 정확한 카디널리티 추정이 얼마나 중요한지를 실제 의료 응용에서 입증한다.

IoMT(Internet of Medical Things) 환경에서는 병원·웨어러블 기기·원격 진료 시스템이 대량의 **멀티모달 의료 데이터**를 생성한다:

- X-ray, CT, MRI 같은 **이미지 데이터** (2D/3D)
- 심전도(ECG), 맥박 센서 같은 **시계열 데이터** (시간 의존적)
- 의사 노트, 처방전, 임상 검사 결과 같은 **텍스트 데이터** (비정형)
- 나이, 성별, 기저질환 같은 **구조화된 메타데이터** (정형)

이 데이터를 기반으로 LLM이 정확한 의료 보조 판단을 내리려면, **검색 증강 생성(RAG)** 파이프라인이 필수적이다. 하지만 기존 RAG 시스템에는 세 가지 핵심 문제가 있다:

문제 1 — 데이터 신선도(Freshness):

의료 데이터는 실시간 업데이트되지만, RAG에 제공되는 검색 결과가 오래된 데이터일 경우 LLM의 판단이 부정확해진다.

예를 들어:

- 환자의 가장 최근 혈액 검사 결과(오늘)가 아닌 3개월 전 결과를 참조하면
- 잘못된 진단으로 이어질 수 있다 (예; 수치가 개선되었는데 악화된 것으로 판단)

문제 2 — 보안과 신뢰:

의료 데이터는 극도로 민감하다. HIPAA, GDPR 같은 규제를 만족하면서도:

- 여러 병원이 데이터를 공유할 때, 중앙 기관 없이 어떻게 신뢰를 확보할 것인가?

[3] Tree-Based RAG-Agent Recommendation System; A Case Study in Medical Test Data 총정리

의료 검사 추천 시스템의 핵심은 "환자의 증상을 보고, 어떤 진료과에 보낼지, 그리고 어떤 검사를 처방할 것인가"라는 결정이다. 이것은 단순하지 않은 **계층적 추론**을 요구한다. HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation) 시스템은 트리 구조로 이를 구현하는데, 각 노드는 전문화된 RAG 에이전트이고, 상위 계층부터 하위 계층으로 점진적으로 검색 공간을 좁혀나간다.

이 구조를 SQL VAQ로 표현하면, 각 트리 레벨에서 카디널리티가 크게 달라지는 **다단계 조인과 벡터 범위 검색의 결합**이 된다. 정확한 카디널리티 추정이 없으면, 상위 레벨의 부정확한 카디널리티가 하위 레벨로 전파되어 최악의 경우 지수적 성능 저하를 초래한다.

본 논문과의 관계: 의료 도메인에서 트리 구조의 각 노드별로 서로 다른 카디널리티 특성을 갖는 VAQ가 발생하며, 이것이 정확한 카디널리티 추정의 필요성을 강하게 보여준다.

의료 검사 추천 시스템에서 "환자의 증상을 보고, 어떤 진료과에 보내고, 어떤 검사를 처방할 것인가"라는 결정은 결코 단순하지 않은 **계층적 추론**을 요구한다.

기존 LLM의 한계:

일반 목적(General-Purpose) LLM에 "환자가 심한 두통을 호소한다"고 입력하면:

GPT-4의 응답:

"신경학적 검사(neurological exam)와 함께,
필요시 MRI나 CT 스캔을 추천합니다.
또한 안과 검진도 고려하세요."

문제점:

1. **과도한 추천:** 모든 가능성을 다루려고 하여 불필요한 검사까지 권장 (비용 증가)
2. **진료과 혼동:** 신경과, 안과, 이비인후과 중 어디가 1차 진료과인지 불명확

[4] Accelerating Machine Learning Inference with Probabilistic Predicates 총정리

이 논문은 "비구조화 데이터에 대한 ML 추론 쿼리에서 비싼 UDF(User-Defined Function)를 어떻게 최적화할 것인가"라는 문제를 다룬다. Exqutor와는 다른 각도에서 ML+DB 최적화를 접근하지만, 두 논문 모두 **"ML/벡터 연산을 관계형 쿼리 옵티마이저에 어떻게 통합할 것인가"**라는 큰 질문을 공유한다.

핵심 아이디어는 비싼 ML 모델(딥러닝 기반 객체 탐지, 얼굴 인식 등) 이전에 **경량 이진 분류기를 "조기 필터"로 삽입**하는 것이다. 이를 통해 불필요한 데이터를 미리 걸러내고 비싼 ML 추론의 횟수를 크게 줄인다. 여러 필터를 계단식으로 연결(cascade)하면 각 단계에서 점진적으로 데이터가 정제되어 전체 비용을 극적으로 감소시킨다.

Exqutor가 벡터 검색의 **카디널리티**를 정확히 추정하여 옵티마이저를 돕는 것과 유사하게, 이 논문은 ML 추론의 **비용과 선택도**를 옵티마이저에 통합한다. 두 논문의 공통 메시지는 **"ML 연산의 특성을 옵티마이저에 정확히 반영하면 극적인 성능 향상이 가능하다"**는 것이다.

본 논문과의 관계; 비싼 연산(벡터 검색 vs. ML 추론)의 특성을 옵티마이저에 통합하는 서로 다른 접근법을 보여주며, 두 방식의 결합 가능성을 시사한다.

비디오 분석, 이미지 검색 같은 ML 추론 쿼리에서 핵심 병목은 **비싼 UDF(User-Defined Function)**이다.

```
SELECT * FROM video_frames
WHERE object_detector(frame, 'red SUV') = TRUE -- 매우 비쌌!
AND timestamp BETWEEN '2024-01-01' AND '2024-01-31';
```

여기서 `object_detector()` 는 딥러닝 모델로:

- ResNet 또는 YOLO 같은 대규모 신경망
- 한 프레임당 수십~수백 ms가 소요됨 (GPU 사용 기준)
- 100만 프레임 비디오라면; $100만 \times 100ms = 100,000초 = \text{약 } 28시간$ 소요

그런데 실제로 "빨간 SUV"가 나오는 프레임은 전체의 **0.1%도 안 될 수 있다** (예; 차가 10,000개 장면 중 10장에만 나타남).

[5] Analytical Engines with Context-Rich Processing: Towards Efficient Next-Generation Analytics 총정리

이 논문은 [6]번 논문(Context-Enhanced Relational Operators with Vector Embeddings)의 **전신이 되는 비전 논문**으로, 임베딩을 관계형 대수에 통합하는 아이디어의 출발점이다. 비전 논문(Vision Paper)이란 구체적인 실험보다는 **새로운 연구 방향을 제시하고 커뮤니티의 논의를 유도**하는 논문으로, [6]의 기술적 기여를 이해하기 위한 철학적·개념적 배경이 된다.

핵심 메시지는 현대 데이터의 **80~90%가 비구조화 데이터**인데, 관계형 DBMS는 여전히 구조화 테이블에 최적화되어 있다는 것이다. 따라서 **임베딩 연산자(E-Operator)**를 관계형 대수의 공식 연산자로 정의하면, 기존 최적화 규칙(선택 푸시다운, 조인 재정렬 등)을 임베딩 연산에도 자동으로 적용할 수 있다. 이것이 바로 VAQ(Vector-Augmented Analytical Query) 최적화의 이론적 토대가 된다.

본 논문과의 관계; 비전 논문으로서 구체적인 실험은 없지만, "임베딩을 관계형 대수에 통합하면 어떤 이득을 얻을 수 있을까"라는 근본적 질문에 대한 답을 제시하며, Exqutor가 이를 실행하는 데 필요한 정확한 카디널리티 추정의 필요성을 이론적으로 정당화한다.

과거(2010년대 초):

데이터 구성:

- 구조화: 80% (숫자, 날짜, 범주형)
- 비구조화: 20% (이미지, 텍스트)

도구 분포:

- RDBMS 투자: 80% (SQL, 트랜잭션, ACID)
- 비구조화 처리: 20% (특수 시스템)

현대(2020년대):

- 구조화: 10~20% (메타데이터만 해당)
- 비구조화: 80~90% (텍스트, 이미지, 오디오, 비디오)

도구 분포:

[6] Context-Enhanced Relational Operators with Vector Embeddings

이 논문은 벡터 임베딩을 관계형 데이터베이스 연산에 공식적으로 통합하는 이론적 기초를 제공한다. 핵심 기여는 **임베딩 연산자(E-Operator)**와 **컨텍스트 강화 조인(E-Join)**을 관계형 대수의 정식 연산자로 정의하고, 이를 기반으로 한 최적화 기법들을 제시하는 것이다.

논문의 중심 질문은 "구조화 데이터(숫자, 날짜)와 비구조화 데이터(텍스트, 이미지)를 같은 쿼리 안에서 의미 기반으로 분석하려면 어떻게 해야 하는가?"이다. 기존 SQL은 정확한 일치(exact match)나 범위 비교에만 적합했지만, 임베딩 연산자를 통합하면 "의미적으로 유사한" 데이터를 찾을 수 있게 된다.

핵심 기술 기여:

1. **프리페치 최적화:** 각 튜플의 임베딩을 한 번만 계산하여, 모델 호출 비용을 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 감소
2. **텐서 조인:** Nested Loop Join을 행렬 곱셈으로 변환하여 약 100배의 속도 향상 달성
3. **인덱스 vs. 스캔 분석:** 선택도에 따라 벡터 인덱스 사용의 효율성이 달라진다는 발견

본 논문과의 관계: Exqutor는 이 논문의 이론적 기초 위에서, "그렇다면 실행 계획을 세울 때 벡터 연산의 카디널리티를 어떻게 정확히 추정할 것인가?"라는 실용적 문제를 해결한다. 특히 인덱스 조인 vs. 스캔 조인의 선택이 선택도에 따라 달라진다는 이 논문의 발견은, Exqutor에서 정확한 카디널리티 추정이 왜 중요한지를 직접 입증한다.

현대 데이터 분석에서는 **구조화 데이터**(숫자, 날짜 등)와 **컨텍스트 풍부한 데이터**(텍스트, 이미지, 오디오 등)를 함께 분석해야 한다. 기존 관계형 연산자(SELECT, JOIN 등)는 정확한 일치(exact match)나 범위 비교에만 적합하고, "의미적으로 유사한" 데이터를 찾는 데는 부적합하다.

예를 들어, 두 테이블에서 "비슷한 상품 설명"을 기준으로 조인하고 싶다면:

```
-- 이런 쿼리는 기존 SQL로 표현 불가
SELECT * FROM table_a JOIN table_b
ON semantic_similarity(a.description, b.description) > 0.8
```

현재는 개발자가:

[7] SingleStore-V: An Integrated Vector Database System in SingleStore

이 논문의 핵심 기술 기여는 **Top() 물리적 연산자**로, ORDER BY + LIMIT 패턴을 인식하여 테이블 스캔 단계에서 직접 top-k 를 찾는 방식이다. 또한 **세그먼트 단위 인덱스 구조**를 통해 분산 환경에서의 확장성을 확보하고, **플러그형 벡터 인덱스**로 Faiss, hnswlib 등 다양한 인덱스를 동적으로 선택할 수 있도록 설계했다.

핵심 기술:

1. **Top() 연산자:** ORDER BY + LIMIT의 효율적 처리
2. **세그먼트 기반 인덱스:** 불변 세그먼트마다 독립적 인덱스, 병렬 검색
3. **플러그형 인덱스:** Faiss, HNSW 등 교체 가능한 인덱스 설계

본 논문과의 관계: SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. 반면 Exqutor는 **여러 테이블에 걸친 조인 순서, 조인 방식, 전체 실행 계획 최적화**에 집중한다. Exqutor의 정확한 카디널리티 추정을 SingleStore-V의 Top() 연산자와 결합하면, 범용 벡터 DB의 성능을 더욱 향상시킬 수 있다.

- **마스터-리프(Master-Leaf) 분산 구조:** 마스터 노드가 조정, 여러 리프 노드에서 데이터 저장 및 처리
- **행 저장소(Rowstore) + 열 저장소(Columnstore) 이중 저장:** 트랜잭션은 행 저장소에, 분석은 열 저장소에서 처리
- **불변 세그먼트(Immutable Segment):** 행 저장소의 데이터가 주기적으로 세그먼트화되어 열 저장소로 이동
- **샤딩(Sharding):** 데이터를 해시 키 기준으로 여러 리프 노드에 분산

SingleStore-V는 이 SingleStore에 벡터 검색 기능을 통합한 것으로, **범용 벡터 데이터베이스**의 대표 사례이다. Milvus 같은 전문 벡터 DB와 달리, 기존 SQL 쿼리와 벡터 검색을 하나의 SQL 문에서 처리할 수 있다.

SingleStore-V의 가장 중요한 기술적 기여는 **Top() 물리적 연산자**이다. 이것이 어떻게 문제를 해결하는지 이해하기 위해, 먼저 기존의 비효율적인 방식을 살펴보자.

기존 방식에서 top-k 벡터 검색을 SQL로 표현하면:

```
SELECT product_id, name, distance
```

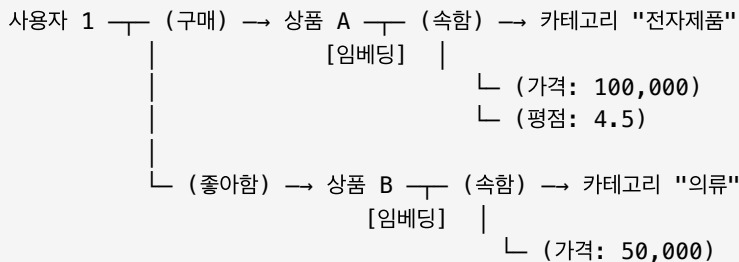
[8] High-Throughput Vector Similarity Search in Knowledge Graphs

지식 그래프는 현대 AI 응용의 핵심 인프라이다:

구조:

- **노드(Entities):** 상품, 사용자, 카테고리 등. 각 노드는 텍스트/이미지 임베딩을 가짐
- **엣지(Relations):** 노드 간 관계. 구조적 속성을 표현 (예: "상품 A는 카테고리 B에 속함")
- **속성(Attributes):** 가격, 평점, 설명 등의 메타데이터

예: 전자상거래 지식 그래프



[9] AnDB: Breaking Boundaries with an AI-Native Database for Universal Semantic Analysis

AnDB는 **AI 네이티브 데이터베이스**의 최신 동향을 보여주는 시스템으로, SQL 자체를 시맨틱 토큰으로 확장하여 비구조화 데이터에 대한 **선언적 시맨틱 분석**을 가능하게 한다.

핵심 아이디어는 전통적인 "SQL로 어떻게 벡터 검색을 표현할 것인가?"라는 문제에서 벗어나, "SQL 자체를 의미 기반 연산에 맞게 재설계하자"는 것이다. SQL에 3가지 시맨틱 토큰(PROMPT, SEM_MATCH, SEM_GROUP)을 추가하여, LLM 호출, 시맨틱 유사도 매칭, 시맨틱 그룹핑을 자연스럽게 표현할 수 있다.

핵심 기술:

1. **PROMPT 토큰**: SQL 내에서 LLM을 직접 호출
2. **SEM_MATCH 토큰**: 선언적 시맨틱 유사도 매칭
3. **SEM_GROUP 토큰**: 시맨틱 유사성 기반 그룹핑
4. **비용-정확도 옵티마이저**: LLM 호출 비용과 추론 정확도를 균형 맞춰 실행 계획 선택

본 논문과의 관계: Exqutor가 "기존 SQL에 벡터 검색을 추가"하는 보수적 접근이라면, AnDB는 "SQL 자체를 AI 연산에 맞게 재설계"하는 급진적 접근이다. AnDB 같은 시스템이 발전할수록, SEM_MATCH와 SEM_GROUP의 카디널리티를 정확히 추정해야 하므로, Exqutor 스타일의 카디널리티 추정이 **더욱 필요**해질 것이다.

현대 데이터는 **구조화 데이터(Structured)**와 **비구조화 데이터(Unstructured)**의 혼합이다:

구조화 데이터 (90년대 중심):

- 숫자: 매출, 수량, 나이
- 날짜: 주문 날짜, 생일
- 범주: 카테고리, 상태
- SQL로 정확히 분석 가능

비구조화 데이터 (현재의 90%):

[10] VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

VBASE는 Exqutor가 직접 통합한 3개 시스템 중 하나이며, Exqutor 논문의 실험에서 **가장 극적인 성능 향상(최대 10,000 배)**을 보인 시스템이다. 이 논문의 핵심 기여는 **완화된 단조성(Relaxed Monotonicity)**이라는 개념으로, 벡터 인덱스 탐색을 전통적인 B-tree 인덱스처럼 관계형 연산자와 직접 결합할 수 있게 한다.

핵심 통찰:

- 전통적 DB 인덱스(B-tree)는 "다음 노드가 반드시 더 나쁠 것"이라는 **엄격한 단조성**을 만족
- 벡터 인덱스(HNSW)는 엄격한 단조성은 없지만, "충분히 탐색하면 더 좋은 결과가 나올 확률이 매우 낮아진다"는 **완화된 단조성**을 만족
- 이를 이용하면 TopK 고정값 없이도 동적으로 탐색을 종료할 수 있고, 벡터+관계형 쿼리를 Volcano 모델에 통합 가능

핵심 기술:

1. **완화된 단조성 검사**: 통계적 성질을 기반으로 탐색 종료 판정
2. **Volcano 모델 통합**: 벡터 인덱스를 표준 관계형 DB 실행 엔진에 직접 삽입
3. **필터링된 유사도 검색**: 벡터 조건 + 관계형 필터를 한 번에 처리

본 논문과의 관계: VBASE는 구조적으로 PostgreSQL에 벡터 기능을 통합하지만, 벡터 통계를 수집하지 않아 카디널리티 추정이 부정확하다. Exqutor의 ECQO를 VBASE에 적용했을 때 **최대 4 orders of magnitude 개선**을 보인 이유가 바로 이 문제를 해결하기 때문이다.

단조성이란? 데이터 구조를 탐색할 때, "다음에 방문할 데이터가 현재보다 '더 나쁜' 결과를 줄 것이라면, 탐색을 멈춰도 된다"는 성질이다.

전통적인 관계형 DB의 B-tree 인덱스는 이 단조성을 **완벽히 만족한다**:

예: 가격이 < 1000인 상품 찾기
B-tree 인덱스 (가격순 정렬):

[11] Milvus: A Purpose-Built Vector Data Management System

- 1. 사용하기 쉬운 인터페이스:** 다양한 응용 분야의 개발자가 쉽게 벡터 검색을 활용할 수 있도록 파이썬, Java, Go 등 여러 언어의 SDK를 제공한다. 개발자는 복잡한 인덱스 구조를 이해할 필요 없이, 간단한 API(search, insert, delete)만으로 벡터 검색 시스템을 구축할 수 있다.
 - 2. 이기종 컴퓨팅 최적화:** CPU와 GPU를 효율적으로 활용하여 성능을 극대화한다. 벡터 검색은 높은 병렬성을 가지므로, GPU의 수천 개 코어를 활용하면 성능을 10배 이상 향상시킬 수 있다. Milvus는 이러한 GPU 가속을 자동으로 처리하므로, 개발자는 GPU 프로그래밍 없이도 GPU의 이점을 누릴 수 있다.
 - 3. 단순 유사도 검색을 넘어선 고급 쿼리:** 속성 필터링(메타데이터 조건), 다중 벡터 검색(여러 벡터 필드 동시 검색), 범위 검색(거리 임계값) 등 실무에서 필요한 고급 기능을 지원한다.
 - 4. 동적 데이터 관리:** 실시간으로 데이터가 삽입·수정·삭제되면서 동시에 검색 요청을 처리해야 하는 상황을 안정적으로 지원한다. 인덱스를 재구축하지 않으면서도 새로운 데이터를 즉시 검색 가능하게 만드는 것이 핵심이다.
 - 5. 확장성과 가용성:** 분산 환경에서 데이터와 쿼리 처리를 여러 노드에 분산시켜, 수십억 건의 벡터를 저장·검색할 수 있어야 한다. 동시에 노드 장애 시에도 서비스가 중단되지 않도록 복제(replication)를 지원해야 한다.
- Milvus는 **클라우드 네이티브 아키텍처**를 채택하여 다음 특징을 갖는다.

저장과 연산의 분리:

- 6. 저장소:** Amazon S3, Google Cloud Storage 같은 객체 스토리지를 사용하여 고가용성과 확장성을 확보한다. 데이터는 스토리지에 영구적으로 저장되며, 여러 노드에서 동시에 접근할 수 있다.
- 7. 연산:** 상태 없는(stateless) 컴포넌트들이 탄력적으로 확장·축소된다. 쿼리 처리 노드는 필요에 따라 동적으로 추가하거나 제거할 수 있으므로, 트래픽 변동에 자동으로 대응할 수 있다.

세그먼트 기반 데이터 관리:

- 8. 데이터는 컬렉션(Collection) 단위로 관리되며,** 이는 관계형 DB의 테이블에 해당한다.
- 9. 컬렉션은 여러 세그먼트(Segment)로 나뉘며,** 이는 물리적 저장 단위이다. 각 세그먼트는 불변(immutable)이므로, 한번 생성되면 수정되지 않는다.
- 10. 세그먼트 단위로 독립적인 인덱스를 구축하고 검색을 수행한다.** 새 데이터가 들어오면 새 세그먼트가 생성되고, 백그라운드에서 정기적으로 여러 세그먼트를 병합한다.

조절 가능한 일관성(Tunable Consistency):

- 11. 로그 백본(log backbone):** 모든 쓰기 작업을 순서대로 로그에 기록하여, 분산 시스템에서 데이터 일관성을 관리한다.
- 12. 사용자는 일관성-성능 트레이드오프를 제어할 수 있다:**

[12] Qdrant: High-Performance Vector Search at Scale

Qdrant의 가장 핵심적인 기술적 기여는 벡터 유사도 검색(HNSW)과 메타데이터 필터링을 **동시에** 수행하는 구조이다.


```

client.search(
    collection_name="products",
    query_vector=[0.1, 0.2, ...],
    query_filter=Filter(
        must=[FieldCondition(key="price", range=Range(lt=1000))]
    ),
    limit=10
)

```

필터 선택도에 따른 자동 전략 전환:

Qdrant의 핵심 통찰은 필터의 선택도(어느 정도 비율의 데이터가 필터를 통과하는가)에 따라 최적의 검색 전략이 달라진다는 것이다.

1. **선택도 높음** (결과 많음, 예: 80%의 데이터가 조건을 만족):
 - HNSW 그래프를 탐색하면서 조건을 불만족하는 노드를 건너뛴
 - 거의 모든 데이터가 조건을 만족하므로, 필터를 먼저 적용하는 것이 비효율적

[13] pgvector: Open-Source Vector Similarity Search for PostgreSQL

왜 이것이 문제인가?

PostgreSQL의 옵티마이저는 각 조건의 선택도(결과의 몇 %가 그 조건을 통과하는가)를 기반으로 **조인 순서, 조인 방식, 인덱스 사용 여부**를 결정한다.

예시 쿼리:

```

SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.1  -- 벡터 조건
    AND category = 'electronics'          -- 스칼라 조건
ORDER BY price;

```

시나리오 1: 벡터 조건의 실제 선택도 = 0.1% (1000개 중 1개만 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 0.1%

이 경우 옵티마이저는 **HNSW 인덱스를 사용하지 않고 Sequential Scan**을 선택할 수 있다 (예상 결과가 많으니 풀 스캔이 빠르다고 판단). 하지만 실제로는 인덱스를 사용하는 것이 훨씬 빠르다.

시나리오 2: 벡터 조건의 실제 선택도 = 90% (거의 모든 데이터가 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 90%

[14] ClickHouse: Lightning Fast Analytics for Everyone

ClickHouse도 범용 DB에 벡터 검색을 추가한 경우로, 카디널리티 추정 문제를 안고 있을 것으로 예상된다. Exqutor의 ECQO 개념을 ClickHouse에 적용하면 유사한 성능 향상을 기대할 수 있다.

기존 OLAP 엔진의 딜레마:

대부분의 데이터베이스는 다음 두 요구사항 중 하나에 특화된다:

1. OLTP (Online Transactional Processing):

- 소량의 데이터를 빠르게 읽고 쓰기 (예: 주문 삽입, 사용자 조회)
- 트랜잭션 안전성, 동시성 제어 중심
- 예: MySQL, PostgreSQL

2. OLAP (Online Analytical Processing):

- 대량의 데이터를 복잡하게 분석 (예: 월간 판매액 집계, 고객 세분화)
- 쿼리 속도, I/O 효율성 중심
- 예: 기존 Redshift, BigQuery

ClickHouse는 "두 마리 토끼를 모두 잡겠다"는 야망을 가지고 설계되었다:

ClickHouse의 목표:

- **고속 적재**: 초당 수백만 건의 데이터를 스트리밍으로 받아서 적재
- **저지연 분석**: 분석 쿼리를 초 단위(또는 밀리초 단위)로 실행

[15] Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers

RAG(Retrieval-Augmented Generation)란?

RAG는 다음 절차로 동작한다:

사용자 질문 → 검색 (관련 문서 찾기) → LLM 입력 (검색 결과 + 질문) → 생성된 답변

검색 단계가 LLM 응답 품질의 상한선(ceiling)을 결정한다:

- 검색 결과가 정확하면 → LLM이 좋은 답변 생성 가능
- 검색 결과가 부정확하거나 불안정하면 → LLM이 아무리 좋아도 제한된 답변만 가능

단일 검색 전략의 한계:

기존 RAG 시스템은 한 가지 검색 방식만 사용했다:

1. 키워드 검색(BM25)만 사용:

- 장점: 정확한 단어 매칭에 강함 (예: "Apple Inc." 검색 시 "Apple" 회사만 찾을 수 있음)
- 단점: 의미적 유사성을 못 잡음
- 질문: "자동차 사고"
- 찾지 못하는 문서: "교통 충돌", "차량 사건" (의미가 같지만 단어가 다름)

2. 밀집 벡터 검색(KNN)만 사용:

[16] Scalable Similarity Search for Big Data

핵심 기여:

1. **메트릭 공간의 기하학적 성질**: 고차원 공간에서의 거리 분포 특성 분석
2. **M-tree 인덱스 구조**: 메트릭 공간의 분할 기반 동적 인덱스
3. **고차원의 저주와 카디널리티**: 거리 집중(concentration) 현상의 수학적 분석
4. **분산 환경으로의 확장**: 빅데이터 환경에서의 유사도 검색

이 논문은 Exqutor가 통계 기반이 아닌 실제 인덱스 탐색 기반의 카디널리티 추정을 선택한 이유의 이론적 근거를 제공한다.

세 가지 핵심 도전 과제:

1. 확장성(Scalability):

- 데이터가 수십억 건에서 수조 건으로 증가해도 응답 시간이 허용 범위 내여야 한다.
- 기존 풀 스캔(brute-force) 방식은 $O(n)$ 복잡도이므로, 데이터 크기 n 이 1000배 증가하면 응답 시간도 1000배 증가한다. 이는 참을 수 없다.
- 인덱스를 통해 $O(\log n)$ 또는 $O(\log^k n)$ 수준으로 개선해야 한다.

2. 프라이버시 보존(Privacy Preservation):

- 클라우드 아웃소싱 환경에서 원본 데이터는 공개하지 않으면서 유사도 검색을 수행해야 한다.
- 암호화된 데이터에 대한 검색(searchable encryption) 기술이 필요하다.

3. 멀티모달 통합(Multimodal Integration):

[17] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

Spider 1.0의 문제점:

기존 Spider 1.0 벤치마크는 학술 환경의 편의성을 위해 다음과 같이 단순화되어 있었다:

| 특성 | Spider 1.0 | 현실 기업 데이터베이스 |

|-----|-----|-----|

| 테이블 수 | 평균 5~10개 | 수백~수천 개 (대기업은 1만+ 테이블) |

| 컬럼 수 | 평균 ~20개 | 1,000개 이상 (한 테이블만 500개 컬럼) |

| SQL 방언 | SQLite만 | BigQuery, Snowflake, PostgreSQL, T-SQL, Oracle 등 다양 |

| 쿼리 복잡도 | 단일, 간단한 쿼리 | 100줄 이상의 멀티 쿼리 워크플로우 |

| 전처리/후처리 | 불필요 | CRITICAL: CLI 호출, 파일 변환, API 통합 |

| 도메인 지식 | 기본 SQL | 도메인별 비즈니스 로직 필수 |

실제 예시 쿼리:

```
-- 실제 기업 쿼리 (Spider 2.0)
WITH sales_data AS (
  SELECT
    order_id,
```

[18] An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint

이 논문은 벡터 검색과 속성 필터를 동시에 처리하는 효율적인 하이브리드 쿼리 프레임워크 NHQ(Native Hybrid Query)를 제안한다. 기존의 Pre-filtering (필터 먼저 적용) 또는 Post-filtering (검색 먼저 수행) 접근법은 속성 선택도가 낮거나 높을 때 극단적인 성능 저하를 보이는 반면, NHQ는 Navigable Proximity Graph(NPG) 구조를 통해 벡터 근접성과 속성 제약을 동시에 활용함으로써 최대 315배의 성능 향상을 달성한다. 특히 속성 분포의 균일성 여부에 따라 두 가지 변형(NPG-C와 NPG-A)을 제공하여, 다양한 실세계 데이터셋에서 재현율 90% 이상을 유지하면서 높은 효율성을 보장한다. NeurIPS 2023의 발표로 학술적 공신력이 높으며, 필터링된 벡터 검색 분야의 최신 기술을 대표한다.

현대의 데이터 검색 애플리케이션은 단순한 벡터 유사도 검색만으로는 부족하다. 대부분의 실세계 시나리오에서는 **벡터 근접성(vector proximity)**과 **속성 제약(attribute constraints)**을 함께 처리해야 한다. 예를 들어, 이미지 검색 시스템에서는 "이 이미지와 비슷한 이미지를 찾되, 특정 카메라 모델로 촬영된 것만 원한다"는 요구가 흔하다. 문서 검색에서는 "이 텍스트와 의미가 비슷한 문서를 찾되, 특정 시간 범위와 카테고리만 제한하고 싶다"는 다중 필터가 일반적이다. 이런 **하이브리드 쿼리(HQ)** 환경에서 기존의 두 가지 접근법은 근본적인 한계를 보인다:

1. 속성 조건을 먼저 적용: `WHERE category = 'electronics'`
2. 필터링 후 남은 데이터에 대해 ANN 검색 수행

장점:

- 검색 범위가 작아지므로, 작은 필터링 결과 집합에 대해서는 빠르다.

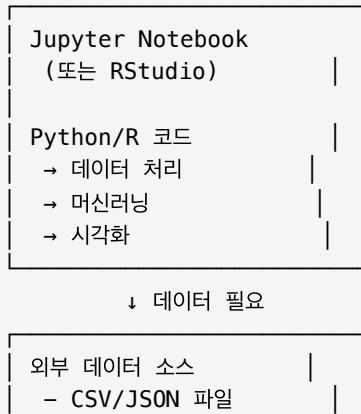
치명적 문제점:

- 속성 조건의 선택도가 낮으면 (예: 1% 미만의 데이터만 통과), 필터링 후 남은 데이터 양이 극소이다.
- 이 극소 데이터셋에 대해서는 벡터 인덱스(예: HNSW)가 효과적으로 작동하지 않는다. 인덱스는 일정 규모 이상의 데이터를 가정하고 설계되었기 때문이다.
- 결과적으로 순차 탐색(linear scan)으로 되돌아가야 하며, 이는 대규모 원본 데이터의 작은 부분만 건너뛰는 것이므로 오버헤드가 크다.
- 또한 별도의 부분 인덱스(partial index)를 구축해야 하므로, 저장 공간과 인덱스 유지 비용이 증가한다.

[19] DuckDB: An Embeddable Analytical Database

현대의 데이터 분석가와 과학자들(주로 파이썬/R 사용자)은 다음과 같은 환경에서 작업한다:

분석 환경:



[20] Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL

역할군: (E) 문제 분석

2. No, 단순 구현 문제다

- 현재 구현이 미흡할 뿐, 개선으로 충분하다
- 벡터 지원은 PostgreSQL에 완벽히 통합될 수 있다

3. Yes and No, 둘 다다

- 일부는 근본적 제약(버퍼 풀, MVCC)
- 일부는 개선 가능(통계, 인덱스 최적화)

이 논문의 대답: 3번. "일부는 근본, 일부는 구현"

문제점: 래치(Latch) 경합

HNSW 탐색의 메모리 접근 패턴:

- 무작위 접근: 순차적이지 않음
- 페이지 접근 빈도: 매우 높음 (초당 수천~수만 번)

PostgreSQL의 버퍼 풀 동시성 제어:

- 각 페이지마다 래치 존재
- 페이지 읽기 전: 래치 획득 (spin lock)

[21] DNA Sequence Classification with Milvus

- 결과 해석 가능 (k-mer 기여도 확인)

1. 쿼리 DNA 서열 입력 |
예: 새로운 박테리아 서열 |

↓

2. 벡터 변환 |
- k-mer 추출 |
- DNABERT 임베딩 |
- 결과: 1536D 벡터 |

↓

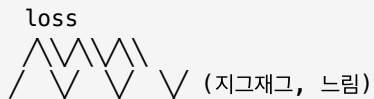
[22] On the Importance of Initialization and Momentum in Deep Learning

신경망을 학습할 때 **확률적 경사 하강법(SGD)**이 느리게 수렴하는 현상이 있다:

이상적 수렴:



실제 SGD 수렴:



원인: 손실 함수의 지형이 방향마다 다르다

[23] New TPC Benchmarks for Decision Support and Web Commerce

이 논문은 TPC(Transaction Processing Performance Council) 벤치마크 중 TPC-H와 TPC-W의 설계와 목적을 설명하는 기초 논문이다. 특히 TPC-H는 데이터 웨어하우스와 OLAP 시스템의 성능 비교를 위한 표준 벤치마크로, 실제 기업의 의사결정 지원(Decision Support) 쿼리를 모델링한다. Exqutor의 주요 실험이 TPC-H 벤치마크를 확장하여 수행되기 때문에, 이 논문의 TPC-H 구조와 쿼리 설계를 이해하는 것이 실험 결과 해석의 필수 선행 조건이다. 벤치마크 설계의 정당성과 데이터베이스 성능 평가의 원칙을 이해하는 데 중요하다.

문제: 데이터베이스 벤더들이 마음대로 성능 주장

예시 (1990년대):

Oracle: "우리 DB가 가장 빠르다"

SQL Server: "우리가 더 빠르다"

PostgreSQL: "우리가 최고다"

증거?

각자 자신에게 유리한 테스트만 수행

다른 DB와 비교하지 않음

테스트 조건을 공개하지 않음

결과:

구매자는 어떤 DB가 실제로 좋은지 알 수 없음

성능 비교가 "입씨름" 수준

[24] The Making of TPC-DS

핵심: 스타/스노우플레이크 스키마

